

User Guide

Spidder — real-time competitive coding battles

Document 3 of 3 · [Technical Approach](#) · [Why/What/How](#)

Part 1 — Running Spidder

Start it

You need **Docker** and nothing else.

```
git clone <repo-url> spidder
cd spidder
docker compose up --build
```

Open <http://localhost:3001>.

First boot takes a few minutes: it builds the images, applies the database schema, seeds the problems, and installs the Python 3.12 runtime into the code sandbox. All of that is automatic. Later starts take seconds.

You'll know it's ready when the logs settle and the page loads.

Stop it

```
docker compose down          # keep the database
docker compose down -v      # wipe the database too
```

If a port is already in use

The usual culprit is **Redis on 6379**, if you already run one locally.

```
cp .env.docker.example .env
echo "REDIS_PORT=6380" >> .env
docker compose up
```

Compose reads `.env` automatically. `POSTGRES_PORT` and `PISTON_PORT` work the same way. Everything has a working default, so only add the lines you need.

For running without Docker, see [SETUP.md](#).

Part 2 — Hosting a battle

Create a room

1. Open <http://localhost:3001>.
2. Enter a display name. That's the whole sign-up — no email, no password.
3. Create a battle. You'll get a **room code** (six characters, e.g. `UVPT2K`).
4. Share that code with your opponent.

Room codes deliberately avoid `0`, `o`, `1`, and `I`, so they're unambiguous when you read one out loud across a room.

Join a room

1. Open the site, enter a display name.
2. Choose to join, and enter the room code.

Anyone with the code can join. There's no invite list — this is designed for people who are already together in a room or on a call.

Start the match

In the lobby, both players:

1. **Pick a side** — Team A or Team B.
2. **Ready up**.

The host can start once both sides are seated, evenly sized, and ready. The server enforces this — you can't start a lopsided or half-ready battle.

A countdown runs, then both sides receive the **same problem** at the same moment.

Part 3 — Playing

The arena

Once the battle starts you'll see the problem statement, an editor, and a timer. Your opponent has exactly the same problem.

Write your solution and submit. Each submission is queued, run against the hidden tests in a sandbox, and the verdict comes back to you.

What you see, and what they see

You see	Your opponent sees
Your full verdict — passed/failed counts	Your passed-count only
Your own code	<i>Nothing</i> of your code

That asymmetry is deliberate. You get enough to feel the race — knowing they're at 4/5 while you're at 2/5 is the whole point — without anything you could use to copy their approach.

The hidden tests are never shown to anyone, including you. You see how many passed, not what they were.

Submitting

You can submit as many times as you like before the clock runs out. Only your **best** submission counts.

How you win

Outcome	What happened
ALL_PASSED	You passed every test — instant win , battle ends
TIMEOUT	Clock expired — highest passed-count wins

If both sides finish the clock on the same passed-count, the tie breaks in favour of whoever **reached that count first**, measured by when the server received the submission.

Practically: if you're stuck at 4/5 with a minute left, keep submitting — an earlier 4/5 beats a later one.

Results

When the battle ends both sides see the final standings: the winner, the reason, and each side's best passed-count. Results are saved, so the record survives a page refresh.

Part 4 — Reference

Ports

Service	URL
Web	http://localhost:3001
HTTP API	http://localhost:4001
WebSocket	<code>ws://localhost:4002/ws</code>
Piston	http://localhost:2000
Postgres	<code>localhost:5432</code> (loopback)
Redis	<code>localhost:6379</code> (loopback)

Current limits

- **1v1 only.** Team modes 2v2–4v4 exist in the data model and rules but are gated in the UI.
- **Python 3.12 only.**
- **Three seeded problems** ship by default: *Sum of Two Numbers*, *Reverse a String*, *Count Vowels*. Each battle is assigned one deterministically from its battle ID, so every player in a given room provably gets the same problem — and a room can be replayed identically.

Adding your own problems

Problems and their hidden tests live in `packages/db/prisma/seed.ts`. Add entries there, then re-run the seed:

```
pnpm --filter @repo/db db:seed
```

To browse or edit the data directly:

```
pnpm --filter @repo/db db:studio
```

Part 5 — Troubleshooting

The page won't load

Give it a minute on first boot — image builds plus provisioning take a while. Then check everything is up:

```
docker compose ps
```

All services should be `running`; `db-init` and `piston-init` should show `exited (0)` — that's success, they're one-shot setup containers.

Submissions hang or every test fails

The sandbox has no language runtime. Check:

```
curl http://localhost:2000/api/v2/runtimes
```

You should see Python 3.12. If the list is empty, `piston-init` didn't finish — check its logs:

```
docker compose logs piston-init
```

"Port is already allocated"

Something else is using one of the ports. See [If a port is already in use](#).

The battle won't start

The server refuses to start an invalid lobby. Confirm both sides are seated, the teams are the same size, and **everyone** has readied up.

I want to start completely fresh

```
docker compose down -v
docker compose up --build
```

`-v` drops the database volume, so the schema and problems are re-seeded from scratch.

Checking service health

```
curl http://localhost:4001/health # -> {"ok":true}
curl http://localhost:4002/health # -> {"ok":true}
```

Reading logs

```
docker compose logs -f          # everything
docker compose logs -f ws-server # one service
docker compose logs -f judge-worker # judging problems
```